

Motion planning with doors makes games hard

Manuel Frohn

RWTH Aachen University, Aachen, Germany
`manuel.frohn@rwth-aachen.de`

Keywords: PSPACE · Doors · Motion Planning · Games

1 Introduction

Many games, such as the Jump'n Runs of the Super Mario series, require the player to move from one point to another to complete a level. This kind of problem is referred to as 1-Player-Motion-Planning. In simple environments, where nothing but paths exist, this problem is easy, as it is equivalent to the well known s-t-Path Problem. But for most games it is not. One of the reasons for this, is the occurrence of doors devices in the games environment. A door device is a gadget, that consists out of three traversable tunnels and an inner state. One of the tunnels opens the door, one closes it and one acts as the "traverse" tunnel. This tunnel may only be traveled if and only if the door is opened. If such a device is part of the environment of a game we know the game to be PSPACE-hard i.e. An algorithm calculating a path as required by a game will do so by using a minimum of Polynomial-Space in relation to the input length.

2 Fundamentals

2.1 The Problem

The problem of concern here is 1-player-motion-planning (1). It is central to understand that 1-player-motion planning on games does not only look at real levels of the game but rather on any level possible in the games implied environment. This means any level is considered, that may be constructed out of the objects making up the game, like platforms, enemies, or general game mechanics such as gravity changes. Comparable, to how the s-t-path problem on general graphs does not only look at certain graphs but rather at all possible general graphs.

Definition 1 (1-Player-Motion-Planning).

1-Player-Motion-Planning is the problem of finding a path from a given point A to a given point B in certain environments (such as a graph or a grid) that can be traversed by the games player.

2.2 The Concern

The main concern of this article is PSPACE-hardness. Just as with time complexity, which concerns itself with the runtime of an algorithm, space complexity concerns itself with the space used by an algorithm. And just as with time complexity, space complexity is divided into different complexity classes. The main one of concern here is the class of PSPACE-hard (2) problems. The problems that require **more** than polynomial space to be solved.

Definition 2 (PSPACE-hardness).

*A problem is PSPACE-hard if and only if, the amount of space a optimal algorithm requires to solve the problem is at **least** polynomial in relation to the input length.*

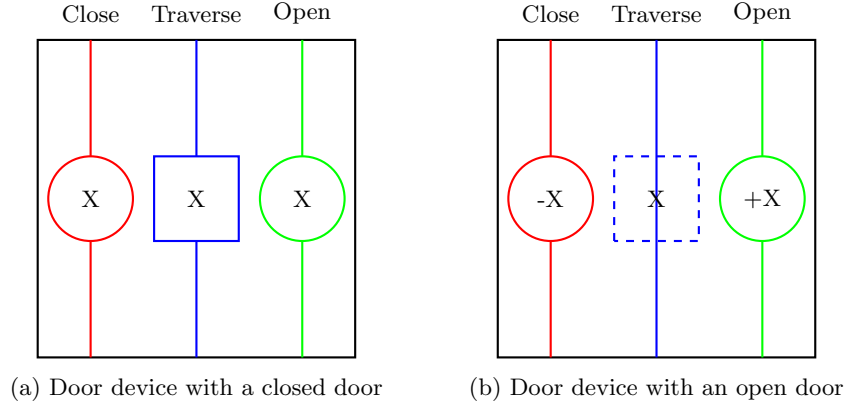
And just as NP has SAT as its canonical problem PSPACE also has a canonical problem, the quantified version of SAT (3) to be precise. There, not only the usage of the logical-or and logical-and is allowed, but also the usage of the quantifiers exists ($\exists$) and forall ($\forall$).

Definition 3 (Quantified-SAT).

Quantified-SAT is the canonical problem of PSPACE. It is an extension of the original SAT problem, expanding the problem by additionally allowing quantified-boolean-algebra expressions. For example $\exists x \forall y : x \wedge y \wedge z$

3 The door device

A door device is a construct made out of three main components: A traverse tunnel, that may only be traversed if the door within it is open, an open tunnel, with a pressure plate which opens the door on being traversed and a close tunnel with a pressure plate which closes the door on being traversed.



4 PSPACE-hardness of doors

To prove PSPACE-hardness for door devices, we will prove a more general theorem(1) concerning doors and their effect on the space complexity of 1-player-motion-planning. This theorem is proven by reducing from True-Quantified-SAT to 1-Player-Motion-Planning with doors. We will then use the theorem to prove PSPACE-hardness for the generic door device, and all the variants, that will be introduced later.

Theorem 1 (Doors are PSPACE-hard).

1-Player-Motion-Planning is PSPACE-hard in an environment that contains doors, that each are controlled by an open and a close pressure plate.

4.1 The reduction

The reduction works by simulating the components of a True-Quantified-SAT problem, and then assembling the complete simulation, out of these components. In the simulation the player has to first pass the quantifier devices upper part, satisfying their condition on the equation, by setting the truth value of the associated variable. After that, the player needs to traverse the clause gadgets, which are only traversable if one of the associated variables is set accordingly to the clause in the original problem. After the player has traversed the clause gadgets, he needs to traverse the lower part of the quantifier gadgets. This serves as a way for the All-Quantifier to test both variable settings, by forcing the player to go through the upper part again (from the current gadget onward, only) with the associated variable being set the other opposite value. Figure 2 shows such a higher level view of the simulation of this True-Quantified-SAT equation:

$$\forall x \exists y \exists z : (\bar{x} \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee z) \wedge (x \vee y \vee \bar{z})$$

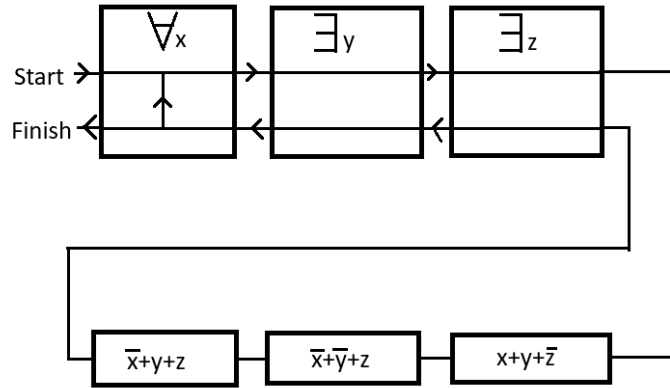
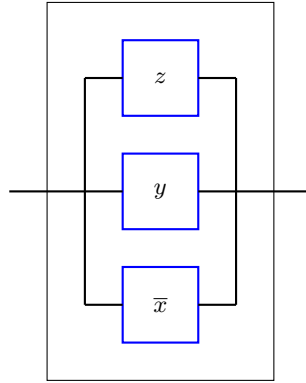


Fig. 2: Simulation of a True-Quantified-SAT equation through 1-Player-Motion-Planning with doors

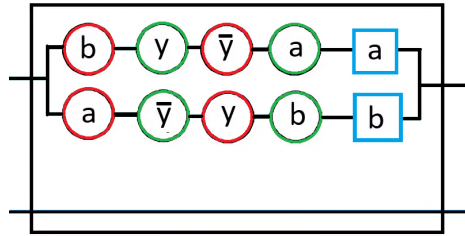
4.2 The clause gadget

The clause gadget is made up of doors, representing the truth values required by the clause to be true. Therefore, if in the quantifier part of the simulation a variable is set as required by the clause, at least one of the doors is open, enabling the player to advance. Creating the behavior we require, as specified in the prior subsection. Figure 3 shows such a clause gadget simulating the clause: $(\bar{x} \vee y \vee z)$

Fig. 3: A clause gadget simulating the clause $(\bar{x} \vee y \vee z)$

4.3 The Exists-Quantifier gadget

The Exists-Quantifier gadget consists out of an upper and a lower part. The upper part itself also consists out of two paths the first one opens the doors associated with one truth value variable of the associated variable, while the other one opens the doors with the inverse truth value. Also, both paths close the doors associated with the opposite truth value of the variable. This creates the behavior of letting the player choose the truth value of the variable, as required by the Exists-Quantifier. Now the lower part serves only as a pass through path, for the agent to get to the finish line or into the next ForAll-Quantifier.

Fig. 4: A Exists-gadget simulation $\exists y$

4.4 The forAll-Quantifier gadget

The forAll-Quantifier gadget consists out of three parts, the upper, the lower and the middle part. The upper when traversed locks, the lower part and sets the variable associated with the gadget to true (by opening the associated doors).

The lower part serves the same purpose as in the Exists-Quantifier just that it contains a door and is therefore only traversable if that door is open. The middle part can only be traversed from the start of the lower part to the end of the upper part. It inverts the variables truth value and opens the lower part. This assembly ensures that both variable truth values have to be tested before the player can proceed onwards to the finish line.

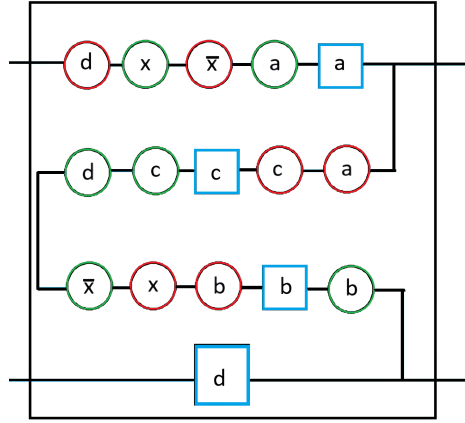


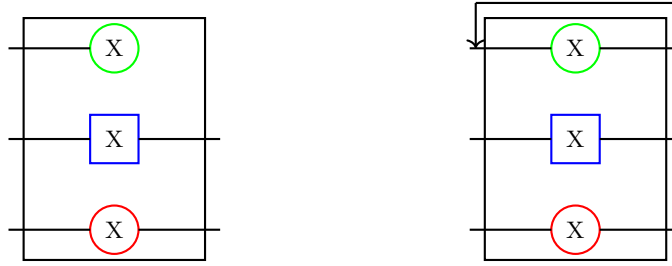
Fig. 5: A Exists-gadget simulation $\forall x$

5 Variants of door devices

For proving PSPACE-hardness of games it comes in handy to have multiple variants of doors at hand, that are easier to model in the games environment. We will introduce three of these advanced door devices, and prove for two of them equivalence to the basic door device in regards to space complexity.

5.1 The open optional

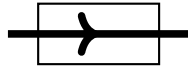
The open optional version loosens the restraint on door devices, that the open tunnel leads from A to B. It allows for a door devices more akin to a door being open by a button, rather than a tunnel that has to be traversed. Equivalence is proven, by constructing the open optional door device out of the basic device as seen in Figure 6 and theorem 1.



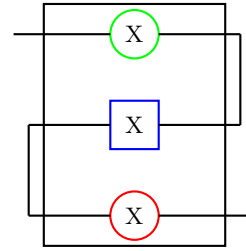
(a) Open optional door device (b) Open optional constructed from basic device

5.2 The directed

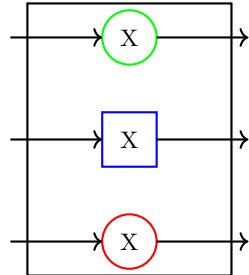
The directed version makes the tunnels, as the name says, directed. Equality is proven again by the theorem and by constructing the directed version out of the basic one. To do so, a helper gadget called "diode" is constructed. It makes a path traversable only from A to B, but not from B to A. It is constructed by chaining together the tunnels of the basic device as seen in Figure 7. With this gadget any tunnel of the door device can be made directional, by putting a diode in front and after the tunnel, as seen in Figure 8.



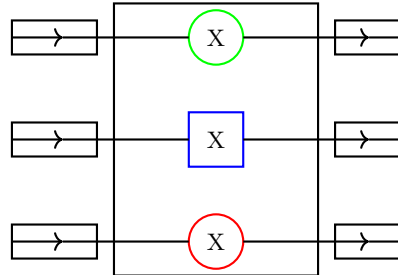
(a) The Diode



(b) The basic device simulating the diode



(a) Directed door device



(b) Simulation of the directed device

5.3 The self closing

The self closing variant merges the close and the traverse tunnel into one, thereby creating a door that closes after being traversed. This variant is also composable with the properties of the prior two variants and comes in a extra variant called symmetric self closing door device. This variant consists only out of two tunnels, each of them containing a door, that closes when the tunnel is traversed and while doing so opening the door in the other tunnel.

6 Application

Now that we have introduced the door device and its variants, it is time to use these gadgets to prove PSPACE-hardness for some games. Therefore we will look at Sokobond, a block pushing game, and a few entries in the 3-D Super Mario series.

6.1 Sokobond

Sokobond is a block pushing game, with a chemical theme. The goal is to form certain molecules out of the atoms and molecules that are on the playing field. The player's movement commands effect is always based on a special marked atom. Molecules are formed when two atoms with free valence electrons are side by side. After the bond is formed, that atom, and everything that is connected to it will also be affected by the movement commands. If no bond can be formed the atom and all that is connected two it may be pushed as a continuous unit. PSPACE-hardness is proven by constructing an open optional door device in the games environment, as seen in Figure 9. The upper corridor serves as the traverse tunnel, which may only be traversed, if the large molecule has been pushed down. This can be achieved, by traversing the left path, which serves as open tunnel. And finally, by traversing the right path and pushing the large molecule up the door is closed, thereby serving as closing tunnel.

6.2 Super Mario Galaxy 2

Super Mario Galaxy 2 is a 3-D platformer, with many mechanics themed around gravity and the empty space. PSPACE-hardness is proven as seen in Figure 10. The leaf raft may be used to cross from one two the other side, but only ever on one side. After the raft has been used it floats trough the electric fence thereby "opening" the other side and "closing" the traversed.

6.3 Super Mario Galaxy

Super Mario Galaxy is the first entry in the series and in many ways similar to the second entry. The PSPACE-hardness prove also uses a symmetric self closing door, though the open switch now is not a leaf raft, but a gravity switch, in the middle of both paths. In combination, with corridors, that are only traversable, if the gravity points in a direction, that makes the other non travelable creates a symmetric self closing door.

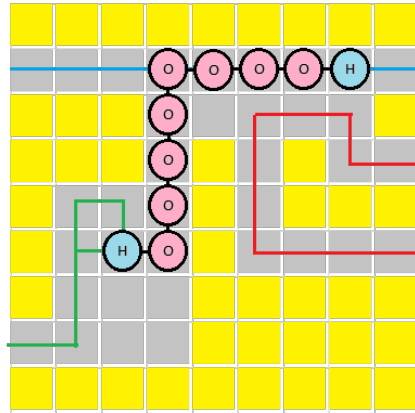


Fig. 9: Sokobond simulating an open optional door device

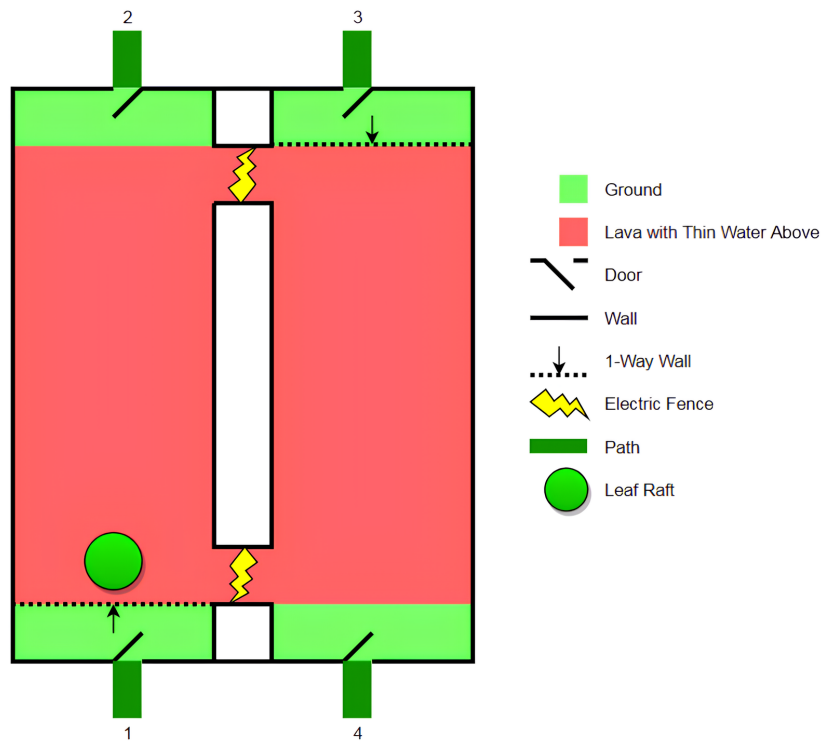


Fig. 10: Super Mario Galaxy 2 simulating a symmetric self closing door. Taken from [2]

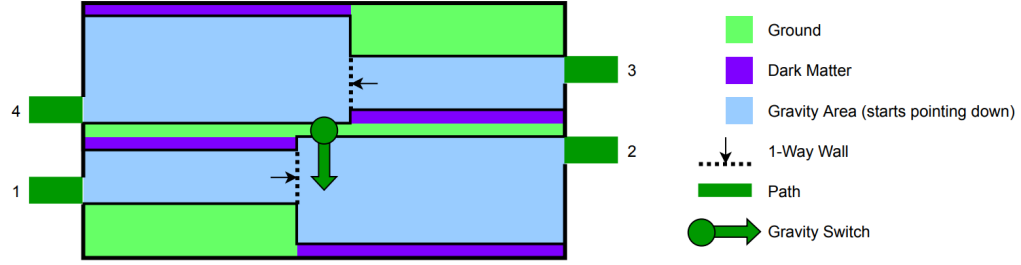


Fig. 11: Super Mario Galaxy simulating a symmetric self closing door. Taken from [2]

References

1. Greg Aloupis, Erik D. Demaine, Alan Guo, and Giovanni Viglietta. Classic nintendo games are (computationally) hard, 2015.
2. Hayashi Ani, Jeffrey Bosboom, Erik D. Demaine, Jenny Diomidova, Dylan Hendrickson, and Jayson Lynch. Walking through doors is hard, even without staircases: Proving pspace-hardness via planar assemblies of door gadgets, 2020.
3. Hayashi Ani, Erik D. Demaine, Dylan H. Hendrickson, and Jayson Lynch. Trains, games, and complexity: 0/1/2-player motion planning through input/output gadgets, 2023.
4. Giovanni Viglietta. Gaming is a hard job, but someone has to do it!, 2013.
5. Giovanni Viglietta. Lemmings is pspace-complete, 2015.